Probability-Augmented Flowchart Prompting: A Bayesian Approach to Uncertainty-Aware Agent Workflows

Anonymous ACL submission

Abstract

Language-model agents are often guided by flowcharts, decision trees, or standard operating procedures (SOPs). These representations constrain task execution, but typically encode branches as deterministic rules and therefore provide little guidance when evidence is incomplete, intent is ambiguous, state changes over time, or action outcomes are uncertain. We introduce Probability-Augmented Flowchart Prompting (PAFP), a framework that represents an SOP as a directed graphical model or dynamic Bayesian network with explicit variables, conditional probabilities, evidence provenance, and action thresholds, and compiles this representation into a structured prompt. We study two questions: (1) whether calibrated probability information improves agent task performance, calibration, safety, or cost, and under what conditions; and (2) how a conventional flowchart can be systematically converted into a probabilistic prompt while preserving business rules and safety boundaries. We provide a conversion procedure and a detailed experimental protocol that separates the effects of probability values, probability-use instructions, and runtime inference. We do not assume that probabilistic prompts are inherently superior to deterministic SOPs: their value depends on probability quality, the agent's ability to use conditional information, and the added reasoning burden. No experimental results are claimed; the proposed hypotheses and protocol are intended to guide empirical evaluation.

Keywords: language-model agents; prompting; Bayesian networks; flowcharts; uncertainty calibration; SOP compilation

1 Introduction

Agent prompts often take the form of flowcharts, step-by-step instructions, or SOPs that specify what to check first, what to do next, and where to branch under particular conditions. Such representations make operational experience executable, but often compress relationships between nodes into deterministic if–then rules. For example, "if the user confirms their identity, continue; otherwise, end the call" does not capture that identity may be supported by multiple pieces of evidence, that recognition tools can be wrong, that a response may be ambiguous, or that continuing under insufficient evidence can be costly.

Bayesian networks offer a natural language for representing these uncertainties: a directed graph encodes dependencies among variables, while conditional probability distributions describe their strengths. When new evidence is observed, beliefs about latent states can be updated. Our central claim is not that a prompt should simply include a list of probabilities. Rather, workflow structure, probability provenance, evidence updates, and action thresholds should be designed together.

We formulate two related but independently testable research questions:

- **RQ1:** Holding tasks, models, tools, and prompt length constant, does providing calibrated probability information improve agent task performance and decision quality? Which task types, probability granularities, and interaction conditions lead to gains or regressions?
- **RQ2:** How can a conventional flowchart/SOP be systematically converted into a probabilistic prompt that preserves operational rules and safety boundaries while updating its state and selecting branches as new evidence arrives?

Our contributions are a conceptual framework, a conversion method, an executable prompt example, and an experimental protocol for validating the framework. We do not report results from experiments that have not been run;

whether probability augmentation improves performance remains an empirical question.

2 Related Work and Positioning

Bayesian networks represent a joint distribution through a directed acyclic graph (DAG) and local conditional distributions, and support probabilistic inference after evidence is observed (Pearl, 1988). Compared with a conventional flowchart, the contribution is not simply to add more branches, but to give branch conditions and state variables compositional probabilistic semantics. For SOPs with feedback and multi-turn interaction, time-indexed variables can form a dynamic Bayesian network, avoiding the treatment of a feedback loop as an ordinary directed cycle.

Research on language-model agents has explored alternating reasoning and action. ReAct interleaves generated reasoning traces with external actions, allowing an agent to revise its plan after observing tool outputs (Yao et al., 2023). PAFP is complementary: it focuses on how uncertain SOP states are represented, how evidence updates them, and when an agent should continue, ask a question, escalate, or stop.

Prior work has examined probabilistic prompting and uncertainty estimation, but addresses different questions. BayesPrompt studies domain abstraction and prompt bias in few-shot inference (Li et al., 2024); Bayesian Prompt Ensembles use multiple prompts to estimate uncertainty in black-box LLM outputs (Tonolini et al., 2024). These studies establish an intersection between prompting and probabilistic methods, but do not directly establish that probabilizing an SOP improves end-to-end agent performance. We treat this as a distinct empirical question.

3 Problem Definition and PAFP Representation

3.1 From Flowcharts to Probabilistic Workflows

Let a conventional SOP be a directed graph $G = (V, E)$, where nodes represent observations, decisions, actions, termination, or escalation, and edges represent control flow. PAFP extends it to a probabilistic workflow $G_p = (V, E, X, \Theta, \Pi, S)$, where X is a set of variables, Θ contains conditional-probability parameters, Π records evidence provenance and update time, and S contains hard safety constraints. Variables may represent external evidence (e.g., user responses or tool outputs), latent states (e.g., intent, identity match, or issue resolution), actions, and outcomes.

$$P(X_1, \ldots, X_n) = \prod_i P(X_i \mid \text{Parents}(X_i)) \quad (1)$$

A conventional workflow edge may encode a logical constraint or a high-probability transition, but these meanings must not be conflated. If an edge means "authorization must be obtained before a charge is made," it should be encoded as a hard constraint. If it means "users who accept an explanation usually accept the proposed solution," it may be represented by a transition probability estimated from data. Hybrid models should explicitly distinguish normative rules from descriptive frequencies.

3.2 Three Probability Types and Their Roles

A model's self-reported "90% confidence" must not be treated as a calibrated probability. Probabilities may come from labeled data, a calibrated classifier, operational statistics, or structured expert elicitation. Each value should include its sample size, applicability, timestamp, and uncertainty interval. If only coarse language (e.g., low/medium/high) is available, it should be treated as ordinal evidence rather than presented as precise statistical measurement.

4 Research Level I: Do Probabilities Improve Agents?

4.1 Mechanisms and Falsifiable Hypotheses

Probability information may help an agent by expressing uncertainty at branch points, supporting posterior-based action selection, and making the trade-off between acting and acquiring information explicit. It may also cause harm: incorrect values can direct the agent to the wrong branch; inconsistent numbers can make an SOP harder to follow; probability tokens can lengthen the prompt; and the model

Table 1: Probability types and their roles in a prompt.

Probability	Meaning	Example	Prompt use
State posterior $P(Z \mid e)$	Belief in a latent state given current evidence	Posterior probability that the user intends to request a refund	Select a clarification question or candidate workflow
Transition $P(Z_{t+1} \mid Z_t, A_t)$	How state changes after an action	Probability that the user accepts a solution after an explanation	Plan the next turn and estimate expected utility
Outcome/risk $P(Y \mid Z, A)$	Possible outcome or risk after an action	False-acceptance probability for identity verification	Select a safety threshold, escalation, or stop action

may ignore numbers or mistake a prior for a posterior.

We therefore state a conditional main hypothesis: when probability parameters are calibrated and match the task environment, the agent receives explicit rules for using them, and the task contains consequential uncertainty, PAFP may improve decision quality over a structurally matched deterministic SOP. The following hypotheses require testing and should not be treated as established findings:

- **H1 (task quality):** On tasks with consequential latent states and evidence updates, PAFP increases task success or reduces decision loss. Gains should be smaller or absent on tasks with little uncertainty.
- **H2 (calibration):** Explicit posteriors and evidence-update instructions reduce agent confidence error, provided that the agent's confidence or probability outputs can be evaluated independently.
- **H3 (information acquisition):** When evidence is insufficient for a high-risk action, PAFP increases useful clarification or tool use and reduces erroneous actions. If clarification is expensive, efficiency may decrease.
- **H4 (sensitivity to probability quality):** Mismatched or stale probabilities can make PAFP worse than a probability-free SOP; transparent provenance and uncertainty intervals may reduce this degradation.

4.2 Conditions and Ablations

Two complementary comparisons are needed. First, hold the graph structure and task fixed and change only the probability information. Second, match prompt-token budgets by comparing probability content with an equally long ordinary business description; this helps prevent an effect of prompt length from being misattributed to probability. Base model, task domain, and probability quality should be treated as stratification factors.

4.3 Task Suite and Evaluation Metrics

A suitable benchmark should contain tasks with traceable states and verifiable outcomes, such as customer-service intent recognition and escalation, troubleshooting, information gathering, appointment changes, or compliance screening. Cases should cover ambiguous evidence, conflicting evidence, missing critical evidence, changing states, rare high-risk situations, and distribution shift. Expert-annotated states and outcomes provide references, while an independent executor or business rules determine whether actions are compliant.

ECE depends on binning and can hide subgroup errors, so it should be paired with Brier score or log loss. High-risk settings should also report false-acceptance rates at operational thresholds and selective risk. For tasks that produce only one final answer, calibration can be estimated across cases; evaluation should not rely solely on the model's self-reported confidence.

4.4 Analysis Plan

The primary analysis should compare conditions on paired cases while controlling model version, temperature, tool access, context length, and sampling policy. Report effect sizes, confidence intervals, and task-stratified outcomes; use a repeated-measures mixed-

Table 2: Experimental conditions and ablations.

Condition	Prompt content	Purpose
C0: deterministic SOP	Original flowchart verbalized as steps, without probabilities	Baseline
C1: probability values	Same structure, with probabilities attached to branch nodes but no explicit update instruction	Isolate the effect of supplying values
C2: probability-use instructions	Conditional probabilities plus instructions to update from evidence before branching	Measure the incremental effect of a probability-use protocol
C3: external inference + prompt	Runtime module computes the posterior and supplies it, its provenance, and interval to the agent	Measure structured probabilistic inference
C4: mismatched probabilities	Controlled perturbation, stale values, or miscalibrated probabilities	Measure fragility and dependence on probability quality

Table 3: Recommended task and evaluation measures.

Dimension	Suggested measures	Interpretation
Task performance	Success rate, goal completion, per-case decision loss	Whether the agent completes the task and the cost of errors or omissions
Calibration	Brier score, log loss, ECE; confidence intervals where possible	Whether predicted confidence matches observed accuracy
Information acquisition	Utility of clarification questions, tool calls, information gain	Whether the agent obtains useful evidence when uncertain
Safety and reliability	Hard-constraint violation, false/ missed escalation, severe error rate	Whether non-probabilistic business boundaries are respected
Efficiency	Latency, tokens, call cost, number of turns	Whether gains justify additional runtime costs

effects model or an appropriate paired test. In addition to overall averages, analyze high- versus low-risk cases, sufficient versus insufficient evidence, and in- versus out-of-distribution tasks. Pre-register primary outcomes and exclusion rules to avoid reporting only favorable subsets.

5 Experimental Setup

This section specifies a prospective protocol for testing RQ1 and RQ2. It is a planned setup, not a report of completed experiments. The selected task domains, models, sample sizes, and probability sources must be instantiated and documented before data collection; no numerical outcome is assumed here.

5.1 Tasks, Cases, and Data Splits

Construct a benchmark from at least two SOP domains with different uncertainty profiles: (i) customer-service troubleshooting and issue-resolution confirmation, and (ii) identity, eligibility, or compliance screening with explicit high-risk constraints. For each domain, represent cases as sequences of observations, tool outputs, latent-state labels, and available actions. Include routine cases, ambiguous or conflicting evidence, missing evidence, state changes after an action, rare high-risk cases, and cases from a later time period or changed population to test distribution shift. Each case should have an adjudicated reference state, acceptable action set, final outcome, and action-specific loss. Two domain annotators independently label the state and acceptable actions; disagreements are adjudicated and reported.

Split cases by user or conversation, never by turn, to prevent leakage. Use a development split to construct prompts and tune thresholds, a held-out in-distribution test split for primary evaluation, and a temporal or population-shifted test split for robustness. Freeze the graph, parameter estimates, prompt templates, and decision thresholds be-

fore evaluating the test splits. Where historical data are used to estimate probabilities, the test period must occur after the estimation period. Report counts by domain, risk tier, evidence sufficiency, and split; do not reuse test labels to calibrate probability sources.

As an initial collection target, annotate at least 1,000 independent cases per domain: 600 for development, 200 for held-out in-distribution testing, and 200 for temporal or population-shifted testing. Where feasible, ensure that each test split contains at least 100 high-risk cases; report any risk-stratified sampling and use appropriate weighting for aggregate estimates. The final sample size should be increased if a paired-case power analysis on development data indicates that this target cannot detect the pre-specified practically meaningful effect.

5.2 Models, Tools, and Probability Sources

Evaluate two instruction-following systems where feasible: one hosted API model and one open-weight model from a different model family. Record the exact model identifier or API version, decoding parameters, system prompt, tool schema, and date of evaluation. Keep tool access, context window, temperature, maximum output length, and retry policy identical across conditions. Use deterministic decoding for the main comparison where supported, and repeat a subset with multiple seeds or samples to estimate run-to-run variance.

For C1–C3, use probabilities from a documented source: a classifier calibrated on the development data, historical frequencies with an appropriate smoothing procedure, or structured expert elicitation when data are insufficient. Record the estimation population, sample size, timestamp, calibration method, and uncertainty interval for each probability. C4 should perturb these values according to a pre-specified protocol (e.g., stale estimates or controlled shifts in log-odds), without changing the underlying cases. C0 receives no probability values. The same source values must be used across all agents and cases so that model-specific estimates are not confounded with the prompt condition.

5.3 Prompt Conditions and Procedure

Implement C0–C4 in Table 2. All prompts should preserve the same SOP graph, node descriptions, safety constraints, and output schema. Match prompt length between C0 and probability conditions by adding neutral operational context to C0 where needed; report the token counts and any residual differences. In C1, present probability values without telling the model how to update them. In C2, add concise instructions to condition on observed evidence and distinguish priors from posteriors. In C3, compute the posterior with an external inference module and supply the current belief state, evidence provenance, and interval; the LLM should not recompute the network. In C4, use the same instructions as C2 or C3 while perturbing probability quality, with the comparison specified in advance.

Run every case under every condition using a paired design. Randomize condition order and anonymize condition labels during outcome adjudication. The agent receives only information available at the current step; future observations and reference labels remain hidden. At each turn, log the observed evidence, posterior supplied (if any), selected node, action, tool calls, output confidence, constraint checks, and final outcome. An independent evaluator scores task completion and compliance from the trace without seeing the condition label. For interactive tasks, use a fixed simulator whose behavior is identical across conditions; separately validate a sample with human users before making claims about deployment performance.

5.4 Outcomes, Sample Size, and Statistical Tests

Pre-register one primary endpoint per domain, such as risk-weighted decision loss or task success under a hard-constraint requirement. Treat calibration, information acquisition, safety violations, latency, token use, and cost as secondary outcomes. Define losses and severity weights with domain experts before test-set evaluation. Estimate sample size from a pilot on development data using the smallest practically meaningful effect and paired-case variance; report the assumptions and power calculation. If the benchmark cannot support

a well-powered test, characterize the analysis as exploratory rather than interpreting a non-significant result as equivalence.

For binary outcomes, use paired tests or mixed-effects logistic regression with condition as a fixed effect and case/domain (and model, when appropriate) as grouping factors. For continuous losses, use a paired bootstrap over cases or a suitable mixed-effects model. Report absolute differences, relative differences where meaningful, 95% confidence intervals, and corrected p-values for planned multiple comparisons. Analyze subgroups specified before evaluation, particularly evidence sufficiency, risk tier, and distribution shift. Report calibration metrics with bootstrap intervals. The study should report all conditions, including regressions and safety failures, and distinguish statistical significance from practical importance.

5.5 Reproducibility and Reporting

Release, subject to privacy and licensing constraints, the anonymized case schema, SOP graphs, prompt templates, probability provenance, evaluation scripts, model versions, and aggregate results. Document any unavailable data and provide synthetic cases only as supplementary examples, not as substitutes for empirical validation. Record deviations from the protocol, failed runs, and exclusions. Do not claim that prompt-level probability use improves performance unless the pre-registered comparisons support that claim across the relevant metrics and risk strata.

6 Research Level II: Converting SOPs into Probabilistic Prompts

6.1 Conversion Pipeline

6.2 Sources of Graph Structure and Parameters

An SOP derived from expert practice can suggest candidate nodes and directional dependencies, but does not by itself establish causality. Domain experts, log analysis, and task mechanisms should jointly review the graph. Conditional probabilities should preferably be estimated from historical data that cover the target population and calibrated on a later-time validation set. For sparse nodes, hierarchical smoothing, prior distributions, or expert estimates may be used, but their provenance and parameter uncertainty must be explicit. If an action changes subsequent state, observational frequencies may be subject to selection bias: $P(\text{outcome} \mid \text{action})$ should not be interpreted as a causal effect without justification. Randomized trials, quasi-experiments, or causal models may be needed.

For an interactive agent, define state Z_t, observation E_t, and action A_t at turn t, and use a dynamic Bayesian network:

$$P(Z_{t+1} \mid Z_t, A_t, E_t), \qquad P(E_t \mid Z_t). \quad (2)$$

At each turn, update beliefs with new evidence and decide whether to continue, clarify, transfer to a human, or terminate. A feedback edge in the flowchart is represented by time indexing; for example, "the user is still unsure after an explanation" means that the next-turn state Z_{t+1} reflects the user's understanding, not that a node points to itself.

6.3 From Posterior Beliefs to Actions

Probabilities alone do not determine actions. The system must specify action utility, cost, and hard constraints. Let a be a candidate action, z a latent state, $U(a, z)$ its utility, and $C(a)$ its cost. The agent may choose the action with the highest expected utility among actions permitted by the constraints:

$$a^* = \arg\max_{a \in \mathcal{A}_S} \left[\sum_z P(z \mid e) U(a, z) - C(a) \right] \quad (3)$$

where $\mathcal{A}_S$ is the set of actions consistent with hard business rules. If policy states that account information must not be disclosed before identity verification, a high posterior match probability cannot override that requirement. High-risk actions may also have explicit risk limits; if every available action exceeds the limit, the agent should transfer the case or stop. Thresholds must follow operational costs and policy, not be inferred from the LLM's self-reported confidence.

7 Probabilistic Prompt Structure and Example

7.1 Recommended Prompt Layers

- **Task and boundaries:** role, goal, applicability, and non-negotiable constraints.

Table 4: A practical SOP-to-prompt conversion pipeline.

Step	Operation	Artifact and checks
1. Parse SOP	Enumerate triggers, decision nodes, actions, termination/escalation paths, and mandatory constraints	Preserve source workflow version; flag ambiguous natural-language conditions
2. Define variables	Rewrite each decision as an observable, latent state, or outcome variable	Variable table: definition, values, observation source, time window
3. Build structure	Identify dependencies from evidence to state, state to action, and action to outcome	DAG; unfold interactive cycles over time
4. Separate rules from probabilities	Mark hard constraints, empirical relationships, unknown relationships, and business preferences	Constraint set S and list of edges requiring estimation
5. Parameterize	Estimate conditional distributions from logs, calibrate classifiers, or conduct structured expert elicitation	Parameters with sample size, scope, version, and intervals
6. Define action policy	Select actions using expected utility/loss, risk limits, and clarification costs	Thresholds, utility table, refusal/escalation exits
7. Compile prompt	Serialize graph, posterior, rules, and output schema into a concise layered prompt	System instructions, current evidence, legal actions, structured output
8. Monitor runtime	Log predictions, observations, actions, and outcomes; audit drift periodically	Audit log, version rollback, human review process

- **Workflow structure:** current node, legal next nodes, and their triggering conditions.
- **Belief state:** current posteriors for key variables, evidence summary, parameter source/version, and uncertainty intervals.
- **Decision rules:** utility or risk thresholds, clarification and escalation conditions; probabilities may select only compliant actions.
- **Execution and logging:** verify evidence, update beliefs, then act; return structured fields for node, evidence, rationale, action, and uncertainty.

7.2 Illustrative Customer-Service Example

The values below illustrate the structure only; they are not empirical results and must not be used directly in production. Suppose an SOP contains a node that asks whether a customer's issue has been resolved; if not, troubleshooting continues or the case is transferred to a human. Let S denote whether the issue is resolved, E whether the user explicitly confirms resolution, and T whether a diagnostic test passes. Before deployment, conditional probabilities should be estimated from labeled data and checked for calibration.

A corresponding prompt fragment could read:

> You are at the "resolution confirmation" node. Update the posterior over S only from observed user responses and tool results; do not invent probabilities. The current probabilities are provided by a calibrated module with provenance and version metadata. If the SOP safety constraints are satisfied and the lower confidence bound of $P(S = \text{resolved} \mid \text{evidence})$ exceeds the business threshold τ, summarize and end. If evidence is insufficient or the interval crosses τ, ask one targeted clarification question. If an escalation condition is met, escalate immediately. Return JSON with the node, evidence, probability source, action, constraint checks, and next step.

This prompt makes the probability source external and explicitly prohibits fabrication. If a deployed system has no calibrated inference component, it may ask the agent to estimate a node probability and provide supporting evidence, but that estimate is not an objective posterior and should be studied separately

Table 5: Illustrative issue-resolution decision.

Field	Illustrative content
SOP node	Determine whether the issue is resolved
Variables	$S \in \{\text{resolved}, \text{unresolved}\}$; $E \in \{\text{confirmed}, \text{unconfirmed/ambiguous}\}$; $T \in \{\text{passed}, \text{failed}, \text{not run}\}$
Evidence	User says "it works now"; diagnostic tool reports a passing test
Posterior example	$P(S = \text{resolved} \mid E = \text{confirmed}, T = \text{passed}) = 0.93$ (illustrative only)
Action policy	If posterior $\geq \tau$ and no unresolved risk remains, summarize and end; if the interval crosses τ, ask a targeted confirmation question; if posterior is low or a high-risk fault is present, continue troubleshooting or escalate
Hard constraints	Do not skip a required safety check based only on positive user tone; do not present model-generated confidence as calibrated probability
Audit output	Current SOP node, evidence, posterior source, selected action, triggered constraints, and need for review

from externally supplied probabilities.

8 Implementation and Governance

Reliability depends on the entire data pipeline, not just prompt wording. Training-data bias, label noise, policy changes, and population differences can miscalibrate probabilities. Each probability field should be associated with model/data versions and an applicability scope; distribution shift should trigger recalibration or fallback to a reviewed deterministic SOP.

- **Avoid false precision:** do not use unsupported decimals; show intervals or coarse levels and mark unknowns when data are sparse.
- **Do not let probabilities override policy:** encode non-negotiable legal, privacy, and business limits as hard constraints.
- **Prevent branch explosion:** probabilize only variables that change action selection; keep other details as ordinary instructions.
- **Prevent feedback contamination:** record agent choices, subsequent evidence, and actual outcomes; do not use the model's own judgment as the sole probability label.
- **Make decisions auditable:** retain evidence summaries, posterior versions, actions, and outcomes; do not use verbose chain-of-thought as the sole audit record.
- **Define failure modes:** if probabilities are missing or stale, evidence conflicts, or inference is unavailable, fall back to a reviewed deterministic SOP or a human workflow.

9 Discussion and Limitations

PAFP extends an SOP from a list of conditional branches to an interface for updating beliefs under uncertain evidence and selecting constrained actions. It does not guarantee that an agent can perform probabilistic reasoning. An LLM may merely repeat values, misuse a marginal probability as a conditional probability, or ignore the applicability of priors and likelihoods. Adding numbers to a prompt can also make an incorrect answer appear more authoritative.

A Bayesian network represents a joint distribution; it does not automatically identify causal effects. If an SOP is intended to determine whether an action caused an issue to be resolved, causal identification is needed rather than a comparison of historical outcomes after different actions. For applications with privacy, financial, or medical consequences, probabilistic decisions must account for asymmet-

ric losses, auditability, and human accountability.

We distinguish two implementations of probabilistic prompting. A text-only implementation serializes graph structure and parameters into the prompt. A hybrid implementation maintains the belief state in an external inference component and supplies only the current posterior and legal actions to the prompt. The latter can make probability updates more consistent, but adds system complexity. Experiments must compare these implementations explicitly so that benefits from a graph-inference component are not mistakenly attributed to prompt wording.

10 Conclusion

We introduce PAFP, a framework that combines flowcharts/SOPs with Bayesian-network mechanisms to guide agents under uncertain states and evolving evidence. Research level I asks whether probability information improves task quality, calibration, safety, and cost, comparing deterministic SOPs, probabilities alone, probability-use instructions, external posterior injection, and mismatched probabilities. Research level II provides a conversion path from SOP parsing through variable definition, graph construction, parameter estimation, constraint modeling, prompt compilation, and runtime auditing. The key design principle is that every probability should have an explicit source and update with evidence, while actions remain governed by both utility and hard constraints. When calibrated probabilities are unavailable, the system should mark uncertainty as unknown rather than invite the model to invent a number. The proposed experimental setup makes these claims testable; empirical validation remains future work.

References

Jiangmeng Li, Fei Song, Yifan Jin, Wenwen Qiang, Changwen Zheng, Fuchun Sun, and Hui Xiong. 2024. Bayesprompt: Prompting large-scale pre-trained language models on few-shot inference via debiased domain abstraction. In *International Conference on Learning Representations*.

Judea Pearl. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann.

Francesco Tonolini, Nikolaos Aletras, Jordan Massiah, and Gabriella Kazai. 2024. Bayesian prompt ensembles: Model uncertainty estimation for black-box large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12229–12272.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.